

CrimeGraph AI — Team Roadmap & Technical Guide

SIH 2026 | 12-Day Sprint Plan



Table of Contents

1. [Team Structure & Ownership](#)
 2. [MVP Scope — What to Build](#)
 3. [Day-Wise Roadmap](#)
 4. [Daily Sync Format](#)
 5. [Technical Deep-Dive](#)
 - 5.1 OCR Pipeline
 - 5.2 NER / Entity Extraction
 - 5.3 Entity Resolution
 - 5.4 Neo4j Graph Schema
 - 5.5 Graph Analytics
-

1. Team Structure & Ownership

#	Role	Person	Owns
1	Backend Dev 1	_____	Data Ingestion + OCR/NLP Pipeline
2	Backend Dev 2	_____	Knowledge Graph + Graph Analytics
3	Backend Dev 3 (Integration Lead)	_____	Core APIs + Auth + Dossier Export
4	Frontend Dev 1	_____	Dashboard + Graph Visualization
5	Frontend Dev 2	_____	Timeline + Map + Reports/Alerts UI

Rule: Har din raat 15-min stand-up (WhatsApp/Discord call). 3 sawaal: Kal kya kiya? Aaj kya karenge? Kahin atke to nahi?

2. MVP Scope — What to Build

✓ Must-Have (build this first, nothing else)

- ☐ File ingestion → OCR → NER/entity extraction
- ☐ Knowledge graph creation (Neo4j)
- ☐ Graph visualization (frontend)
- ☐ Centrality + community detection
- ☐ Basic search/filter
- ☐ PDF report with source provenance

⚙ Should-Have (only if Must-Have done by Day 8)

- ☐ Temporal + geospatial views
- ☐ Alert rules engine
- ☐ Role-based access polish

🌟 Stretch (skip unless way ahead of schedule)

- ☐ GNN-based link prediction
 - ☐ Streaming ingestion (Kafka)
 - ☐ Advanced anomaly detection
-

3. Day-Wise Roadmap

◆ Phase 1: Foundation — Day 1 to 3

Person	Day 1	Day 2	Day 3
Backend Dev 1	Design file upload API (CDR/IPDR CSV, FIR PDF)	Set up OCR pipeline (Tesseract/PaddleOCR)	Integrate spaCy NER — extract names, phones, locations

Person	Day 1	Day 2	Day 3
Backend Dev 2	Neo4j setup, define node/edge schema	Basic graph write API (insert nodes/edges)	Entity resolution v1 (exact + fuzzy match)
Backend Dev 3 (You)	Harden Auth (JWT refresh, roles)	RBAC middleware	Case management API
Frontend Dev 1	Finish Login/Register UI, connect to backend	Dashboard shell — layout, routing	Graph canvas setup (Cytoscape.js, dummy data)
Frontend Dev 2	Align on component library with Dev 1	Timeline UI skeleton (mock data)	Map UI skeleton (Leaflet, dummy markers)

🔗 **Checkpoint (End of Day 3):** Auth works end-to-end, Neo4j accepts data, dashboard shell live, timeline/map skeletons ready.

◆ Phase 2: Core Features — Day 4 to 7

Person	Day 4–5	Day 6–7
Backend Dev 1	Data normalization layer (phone/name cleanup)	Connect NLP output → auto-create graph entities
Backend Dev 2	Graph analytics: centrality, Louvain community detection	Graph query API (search, connections, shortest path)
Backend Dev 3 (You)	Search/filter API (name, date, location, case)	Alert rules engine + start Dossier PDF export API
Frontend Dev 1	Connect graph canvas to real API	Search & filter panel wired to backend
Frontend Dev 2	Connect timeline UI to real data	Connect map to real location data

🔗 **Checkpoint (End of Day 7):** Full data flow — upload → NLP → graph → visible on frontend. Search works. Timeline/map show real data.

◆ Phase 3: Integration & Polish — Day 8 to 10

Team	Day 8	Day 9	Day 10
All Backend	Full pipeline test (upload → OCR → NER → resolution → graph)	Fix integration bugs, finish Dossier export	API docs (Swagger), load test
All Frontend	Alerts panel UI, connect to backend	Dossier export button, UI polish	End-to-end testing with real backend

🎯 **Checkpoint (End of Day 10):** Full working demo — upload → graph → analytics → alerts → dossier, all connected.

◆ Phase 4: Demo Prep — Day 11 to 12

- **Day 11:** Synthetic demo dataset ready (2 fake clusters connected via hidden intermediary). Full team dry-run.
 - **Day 12:** Pitch deck polish, rehearse "before vs after AI" story, fix last-minute bugs, buffer time.
-

4. Daily Sync Format

Keep it under 15 minutes, every evening:

1. **Kal kya complete hua?**
2. **Aaj kya target hai?**
3. **Kisi ko blocker hai kya** (kisi aur ke API/component ka wait)?

Backend Dev 3 is Integration Lead — daily check karo ki teeno backend pieces sahi se jud rahe hain, warna Day 8 pe chaos hoga.

5. Technical Deep-Dive

5.1 OCR Pipeline

Goal: Scanned FIR/case documents se readable text nikaalna, jo NER ko feed hoga.

1. **Input Handling** — PDF ko page-wise images mein convert karo (`pdf2image`)
2. **Preprocessing** — Grayscale → denoise → binarization → deskew (OpenCV)
3. **OCR Engine** — Tesseract (fast, MVP ke liye) ya PaddleOCR (better accuracy, handwritten/poor scans)
4. **Post-processing** — Regex cleanup (phone numbers, dates), confidence score store karo
5. **Output** — Clean text NER pipeline ko jaata hai

```
from pdf2image import convert_from_path

import pytesseract

import cv2

import numpy as np

pages = convert_from_path('fir_document.pdf', dpi=300)

extracted_text = []

for page in pages:

    img = cv2.cvtColor(np.array(page), cv2.COLOR_RGB2BGR)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    _, thresh = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)

    text = pytesseract.image_to_string(thresh, lang='eng')

    extracted_text.append(text)

full_text = "\n".join(extracted_text)
```

Gotchas: Multi-column layouts confuse OCR (split columns first); Hindi documents need `lang='hin+eng'`; always store source doc ID + page number for provenance.

5.2 NER / Entity Extraction

Goal: OCR/text data se structured entities (names, phones, locations, dates, orgs) nikaalna.

1. Use **spaCy** pre-trained model (`en_core_web_trf` for better accuracy) as base
2. Add **custom regex rules** for domain-specific patterns (Indian phone numbers, FIR numbers, vehicle registration formats)
3. For higher accuracy, fine-tune a **Hugging Face Transformer** NER model on a small labeled sample of FIR-style text
4. Output format: structured JSON — `{entity_type, value, confidence, source_doc_id, char_offset}`

```
import spacy
```

```
nlp = spacy.load("en_core_web_trf")
```

```
doc = nlp(full_text)
```

```
entities = [{"text": ent.text, "label": ent.label_} for ent in doc.ents]
```

Gotcha: spaCy's default model won't catch Indian names/phone formats well — combine with regex for phone numbers and a custom entity ruler for local context.

5.3 Entity Resolution

Goal: Same person/number ko alag records mein match karna (aliases, spelling variations, reused SIMs).

1. **Deterministic matching** — exact phone number, exact ID match
2. **Fuzzy string matching** — names (use `rapidfuzz` library, Levenshtein distance)
3. **Phonetic matching** — for name variations (Soundex/Metaphone algorithms)
4. **Confidence scoring** — combine all signals into one score; anything above threshold auto-merges, below threshold flagged for analyst review

```
from rapidfuzz import fuzz
```

```
similarity = fuzz.ratio("Rakesh Kumar", "Rakesh Kummar") # ~92
```

Gotcha: Never auto-merge on name similarity alone — always require at least one strong signal (shared phone/device/location) alongside fuzzy name match.

5.4 Neo4j Graph Schema

Goal: Typed nodes and edges storing entities + relationships with provenance.

Node types: PERSON, PHONE, DEVICE, ACCOUNT, LOCATION, CASE, EVENT

Edge types (examples):

- PERSON -[:USES]-> PHONE
- PERSON -[:LOCATED_AT]-> LOCATION
- ACCOUNT -[:TRANSFERRED_TO]-> ACCOUNT
- DEVICE -[:SEEN_WITH]-> DEVICE

Every edge should carry properties: source_doc_id, timestamp, confidence, created_by_model_version

```
CREATE (p:PERSON {name: "Rakesh Kumar", case_id: "C001"})
```

```
CREATE (ph:PHONE {number: "98XXXXXXXX"})
```

```
CREATE (p)-[:USES {confidence: 0.92, source_doc_id: "FIR-2026-001"}]->(ph)
```

5.5 Graph Analytics

Goal: Surface key players, communities, and hidden connections.

- **Betweenness centrality** — bridge nodes connecting separate groups
- **PageRank** — most influential/connected nodes
- **Louvain community detection** — clusters/cells within the network

Use **Neo4j Graph Data Science (GDS) library** for large graphs, or **NetworkX** for MVP-scale prototyping.

```
import networkx as nx
```

```
G = nx.Graph()
```

```
# add edges from Neo4j query results
```

```
communities = nx.algorithms.community.louvain_communities(G)
```

```
centrality = nx.betweenness_centrality(G)
```

Share this document with your team via Google Drive/Notion so everyone can check off tasks as they complete them.